Python for Artificial Intelligence

Lambda, Sorting, Higher-Order Functions & Modules

Week 2, Day 2

Today's Agenda

- Hour 1: Lambda functions and sorting with key=
- Hour 2: map() and filter() with lambda
- Hour 3: Organizing code into modules and packages
- By the end: you can write short throwaway functions, sort data your own way, transform and filter collections in one line, and split code across files

Hour 1

Lambda Functions & Sorting with key=

CONCEPT

Why do we need lambda?

- Sometimes you need a tiny function for exactly one use, right where you use it
- Writing a full def function just for a one-line calculation feels heavy
- A lambda is a small, unnamed (anonymous) function you write inline
- It can only contain a single expression, not full statements like if/for blocks
- Rule of thumb: if the function body is one short expression and you won't reuse it by name, lambda is a candidate

def (named function)

```
def square(n):
    return n * n

result = square(5)
```



lambda (anonymous function)

```
square = lambda n:
n * n

result = square(5)
```

lambda Syntax

SYNTAX

```
lambda parameters: expression
```

PARAMETERS

`parameters` zero or more parameter names, comma-separated (just like a def)

`expression` a single expression whose result is automatically returned

YOU CAN

- ✓ Assign it to a variable and call it like any function
- ✓ Take multiple parameters: lambda a, b: a + b
- ✓ Use it directly as an argument to another function

YOU CANNOT

- ✗ Write multiple lines or statements inside it
- ✗ Use if/else blocks, for loops, or print statements inside it
- ✗ Give it a docstring

Returns: the value of the expression, automatically (no return keyword needed)

EXAMPLE

A Lambda vs. the Same def Function

Both do exactly the same job. Lambda is just a shorter way to write it.

```
def add(a, b):
    return a + b

add_lambda = lambda a, b: a + b

print(add(3, 4))          # 7
print(add_lambda(3, 4))   # 7
```

THINK & PRACTICE

Quick Check

Write a lambda called "cube" that takes one number n and returns n cubed (n * n * n). Then call cube(3) and predict the output.

ANSWER

Quick Check: Answer

```
cube = lambda n: n * n * n

print(cube(3))  # 27
```

CONCEPT

Sorting with sorted() and key=

- sorted(iterable) returns a new, sorted list; it does not change the original
- By default it sorts numbers by value and strings alphabetically
- What if you want to sort by something else, like the length of a word, or someone's age?
- The key= parameter tells sorted() what to look at for each item, instead of the item itself
- key= expects a function; a lambda is perfect here because the rule is usually one short expression

sorted() with key=

SYNTAX

```
sorted(iterable, key=function, reverse=False)
```

PARAMETERS

`iterable` the list (or other sequence) you want sorted

`key` a function called on each item; sorting happens on its return value

`reverse` True sorts largest-to-smallest / Z-to-A; default is False

YOU CAN

✓ sorted(words, key=len) to sort by length

✓ sorted(names, key=lambda n: n.lower()) to sort ignoring case

✓ sorted(data, key=lambda x: x[1]) to sort a list of tuples by the second value

YOU CANNOT

✗ Pass key as a value instead of a function (key=len, not key=len())

✗ Expect the original list to change (use list.sort() for that instead)

Returns: a new sorted list

EXAMPLE

Sorting Words by Length

key=len tells sorted() to compare each word's length, not the word itself.

```
words = ["banana", "fig", "apple", "kiwi"]

by_length = sorted(words, key=len)
print(by_length)  # ['fig', 'kiwi', 'apple', 'banana']

by_length_desc = sorted(words, key=len, reverse=True)
print(by_length_desc)  # ['banana', 'apple', 'kiwi', 'fig']
```

EXAMPLE

Sorting a List of Tuples with a Lambda

Each student is (name, score). We sort by score using a lambda as the key.

```
students = [("Ali", 78), ("Sara", 92), ("Bilal", 85)]

by_score = sorted(students, key=lambda s: s[1], reverse=True)
print(by_score)
# [('Sara', 92), ('Bilal', 85), ('Ali', 78)]
```

THINK & PRACTICE

Think & Practice

You have prices = [199, 49, 999, 15, 320]. Use sorted() with a lambda key to sort the list from cheapest to most expensive, and then again from most expensive to cheapest.

ANSWER

Think & Practice: Answer

```
prices = [199, 49, 999, 15, 320]

cheapest_first = sorted(prices, key=lambda p: p)
print(cheapest_first)  # [15, 49, 199, 320, 999]

priciest_first = sorted(prices, key=lambda p: p, reverse=True)
print(priciest_first)  # [999, 320, 199, 49, 15]
```

THINK & PRACTICE

Think & Practice (Harder)

You have a list of dictionaries: products = [{"name": "Pen", "price": 20}, {"name": "Bag", "price": 1500}, {"name": "Book", "price": 350}]. Sort this list so the cheapest product comes first, using a lambda that looks at the "price" key.

ANSWER

Think & Practice (Harder): Answer

```
products = [
    {"name": "Pen", "price": 20},
    {"name": "Bag", "price": 1500},
    {"name": "Book", "price": 350},
]

cheapest_first = sorted(products, key=lambda p: p["price"])
for item in cheapest_first:
    print(item["name"], item["price"])
# Pen 20
# Book 350
# Bag 1500
```

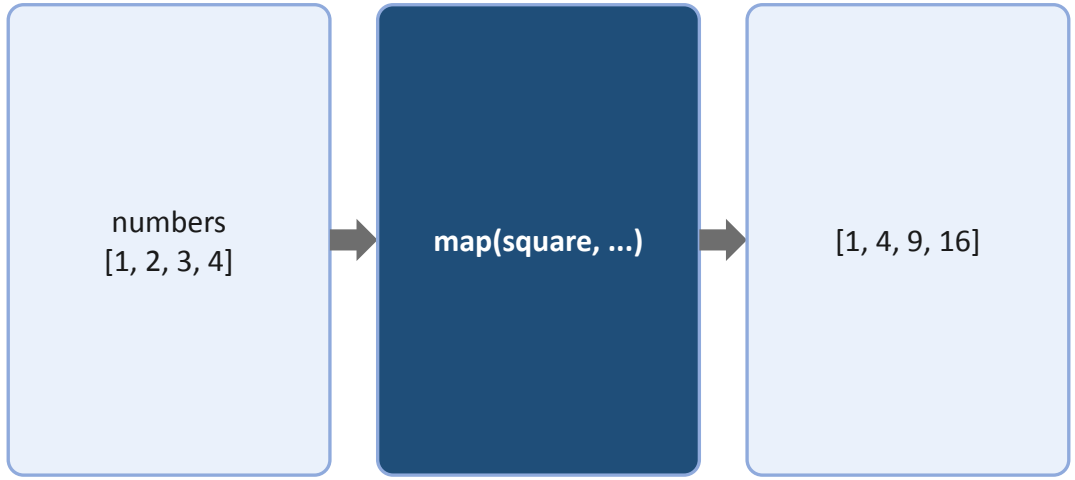
Hour 2

map() and filter() with Lambda

CONCEPT

map(): Apply a Function to Every Item

- map(function, iterable) applies function to every item and gives back the results
- It replaces a common loop pattern: 'for every item, do this transformation'
- map() returns a map object, a lazy iterator, so we usually wrap it in list(...) to see the results
- It pairs naturally with lambda when the transformation is one short expression



map() Syntax

SYNTAX

```
map(function, iterable)
```

PARAMETERS

`function` a function (often a lambda) that takes one item and returns a new value

`iterable` the list (or other sequence) to transform

YOU CAN

- ✓ Wrap the result in list(...) to get an actual list back
- ✓ Use a lambda for a quick one-line transformation
- ✓ Use an already-defined function instead of a lambda

YOU CANNOT

- ✗ Use it to filter items out (that is filter()'s job)
- ✗ Index into the map object directly without converting it to a list first

Returns: a map object (an iterator); convert with list() to see the values

EXAMPLE

Doubling Every Number with map()

Compare the loop version and the map() version side by side.

```
numbers = [1, 2, 3, 4]

# loop version
doubled_loop = []
for n in numbers:
    doubled_loop.append(n * 2)

# map version
doubled_map = list(map(lambda n: n * 2, numbers))

print(doubled_loop)  # [2, 4, 6, 8]
print(doubled_map)   # [2, 4, 6, 8]
```

CONCEPT

filter(): Keep Only What Passes a Test

- filter(function, iterable) keeps only the items where function returns True
- The function is a test, often called a predicate, applied to each item one at a time
- Just like map(), filter() returns a lazy filter object, so wrap it in list(...)
- map() transforms every item; filter() decides which items survive

filter() Syntax

SYNTAX

```
filter(function, iterable)
```

PARAMETERS

`function` a function (often a lambda) that returns True or False for one item

`iterable` the list (or other sequence) to filter

YOU CAN

- ✓ Combine a comparison inside a lambda, like lambda n: n % 2 == 0
- ✓ Wrap the result in list(...) to get an actual list
- ✓ Chain filter() and map() together on the same data

YOU CANNOT

- ✗ Use it to change the values of items (only map() transforms values)
- ✗ Pass a function that doesn't return True or False

Returns: a filter object (an iterator); convert with list() to see the values

EXAMPLE

Keeping Only Even Numbers

The lambda returns True for even numbers, so only those survive the filter.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]

evens = list(filter(lambda n: n % 2 == 0, numbers))
print(evens)  # [2, 4, 6, 8]
```

EXAMPLE

Chaining filter() and map()

First keep the even numbers, then square whatever survived.

```
numbers = [1, 2, 3, 4, 5, 6]

evens = filter(lambda n: n % 2 == 0, numbers)
squared_evens = list(map(lambda n: n * n, evens))

print(squared_evens)  # [4, 16, 36]
```

THINK & PRACTICE

Quick Check

You have words = ["cat", "elephant", "dog", "hippopotamus"]. Use filter() with a lambda to keep only words with more than 3 letters. Predict the output.

ANSWER

Quick Check: Answer

```
words = ["cat", "elephant", "dog", "hippopotamus"]

long_words = list(filter(lambda w: len(w) > 3, words))
print(long_words)  # ['elephant', 'hippopotamus']
```

THINK & PRACTICE

Think & Practice

You have prices = [199, 49, 999, 15, 320]. Use map() with a lambda to apply a 10% discount to every price (multiply by 0.9), then round each result to a whole number using round().

ANSWER

Think & Practice: Answer

```
prices = [199, 49, 999, 15, 320]

discounted = list(map(lambda p: round(p * 0.9), prices))
print(discounted)  # [179, 44, 899, 14, 288]
```

THINK & PRACTICE

Think & Practice (Harder)

You have names = ["ali", "Sara", "BILAL", "zara"]. In one line, first filter out any name that is not entirely lowercase (hint: use str.islower()), then map the survivors to their capitalized form (hint: use str.capitalize()). Print the final list.

ANSWER

Think & Practice (Harder): Answer

```
names = ["ali", "Sara", "BILAL", "zara"]

lowercase_only = filter(lambda n: n.islower(), names)
capitalized = list(map(lambda n: n.capitalize(), lowercase_only))

print(capitalized)  # ['Ali', 'Zara']
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

Hour 3

Modules & Packages

CONCEPT

Why Split Code Into Multiple Files?

- So far, every program has lived in a single .py file
- Real projects grow to thousands of lines; one giant file becomes impossible to navigate
- A module is simply a .py file containing functions, and you can import it into another file
- A package is a folder of related modules, grouped together so they can be imported as a unit
- Splitting code lets you reuse functions across projects without copy-pasting them

import Syntax

SYNTAX

```
import module_name
import module_name as alias
from module_name import function_name
```

PARAMETERS

`module_name` the name of a .py file (without .py) or a built-in module like math

`alias` a shorter name to refer to the module by, e.g. import math as m

`function_name` one specific function or value to pull directly into your file

YOU CAN

✓ import math and then call math.sqrt(16)

✓ from math import sqrt and then call sqrt(16) directly

✓ Import your own file, e.g. import helpers if helpers.py is in the same folder

YOU CANNOT

✗ Import a module that Python cannot find (wrong name, wrong folder)

✗ Use a function from a module before importing it

Returns: nothing directly; it makes the module's functions and values available in your file

EXAMPLE

Using Python's Built-In math Module

math is a built-in module, always available, no installation needed.

```
import math

print(math.sqrt(16))      # 4.0
print(math.pi)            # 3.141592653589793
print(math.floor(7.9))    # 7

from math import ceil
print(ceil(7.1))          # 8
```

EXAMPLE

Writing and Importing Your Own Module

helpers.py contains one function. main.py imports and uses it.

```
# helpers.py
def greet(name):
    return "Hello, " + name + "!"

# main.py (same folder as helpers.py)
import helpers

print(helpers.greet("Ayesha"))  # Hello, Ayesha!
```

CONCEPT

A Note on Packages

- A package is a folder containing multiple module files, plus a special __init__.py file
- The __init__.py file can be empty; its presence tells Python 'treat this folder as a package'
- You already import packages all the time: think of a folder like statistics or random
- For now: know that a project grows from one file, to several modules, to organized packages
- We will use real packages (like NumPy and Pandas) starting next week

THINK & PRACTICE

Quick Check

What is the difference between import math and from math import sqrt? After each one, how would you call the square root function?

ANSWER

Quick Check: Answer

```
import math
math.sqrt(25)       # must use the math. prefix

from math import sqrt
sqrt(25)            # no prefix needed, imported directly
```

THINK & PRACTICE

Think & Practice

Create a file called shapes.py with one function, area_of_circle(radius), that returns math.pi * radius ** 2. Then, in a separate script, import shapes and print the area of a circle with radius 5.

ANSWER

Think & Practice: Answer

```
# shapes.py
import math

def area_of_circle(radius):
    return math.pi * radius ** 2

# main.py
import shapes

print(shapes.area_of_circle(5))  # 78.53981633974483
```

Home Practice: Interview-Style Questions

- 1. Write a lambda that takes two numbers and returns the larger one, without using if.
- 2. Given a list of tuples (name, age), sort it by age using sorted() and a lambda key.
- 3. Given a list of numbers, use filter() and map() together to return the squares of only the odd numbers.
- 4. Explain in your own words: why can't a lambda contain a for loop?
- 5. Create a small module math_utils.py with two functions of your choice, and import and use both from another file.

Key Takeaways & Coming Up

- lambda creates a short, unnamed function for one-off use, especially as an argument to other functions
- sorted(..., key=...) lets you sort by any rule you define, not just default ordering
- map() transforms every item; filter() keeps only items that pass a test; both pair naturally with lambda
- Modules let you organize code across files; packages group related modules together
- Coming up in Day 8: Object-Oriented Programming, classes, objects, and the __init__ constructor